

Aula 03 — Spring Framework

CRUD ã Formulários ã StockSales

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Parte 1 — 18h30 às 20h

- Revisão rápida da Aula 02
- Como funciona uma requisição HTTP
- CRUD: cadastrar, listar, editar e excluir
- Projeto novo: **cadastro-alunos** — fazemos juntos
- Desafios no mesmo projeto

Parte 2 — 20h20 às 22h

- Voltamos ao **StockSales**
- CRUD de produtos e de clientes

Dois projetos nesta aula

Não vamos misturar os códigos.

cadastro-alunos Projeto **novo**, só desta primeira parte. Aqui aprendemos CRUD com um cadastro simples de alunos.

StockSales O sistema da loja, que já existe. Usamos na segunda parte, depois do intervalo.

Quando aparecer um código, o slide vai dizer:

qual projeto e **qual arquivo** estamos mexendo.

Mapa da aula

- 1 Revisão da Aula 02
- 2 O que é uma requisição
- 3 CRUD
- 4 Projeto cadastro-alunos
- 5 Desafios
- 6 Parte 2 — StockSales
- 7 Recapitulando

O que já sabemos

- Criar um projeto no `start.spring.io`
- Página HTML fica em `src/main/resources/static/`
- Dados vêm de um Controller, em JSON
- O JavaScript busca esses dados com `axios.get`

No StockSales, a tela de produtos **só lista**.

Ainda não dá para cadastrar, alterar nem apagar.

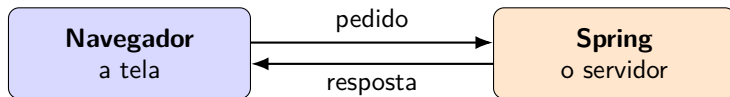
O Spring atende dois tipos de pedido:

- **Página:** / ou /produtos.html
— o arquivo HTML da pasta `static/`
- **API:** /api/produtos
— o método Java do Controller, devolvendo JSON

A pessoa vê a página.

A página pede os dados para a API.

Navegador e servidor conversam



Cada clique que busca ou grava dados faz um **pedido**.

O servidor devolve uma **resposta**.

Esse pedido se chama **requisição HTTP**.

Uma requisição tem três partes principais

Pensem numa carta:

- ❶ **O que você quer fazer** — o verbo:
GET, POST, PUT ou DELETE
- ❷ **Para onde** — o endereço:
`http://localhost:8080/api/alunos`
- ❸ **O conteúdo** — o **corpo** da carta, se houver:
os dados em JSON

GET quase nunca leva conteúdo.

POST e PUT quase sempre levam.

GET: só pergunta

Exemplo: listar alunos.

Pedido que o navegador manda

- Verbo: GET
- Endereço: `/api/alunos`
- Corpo: *vazio* — não precisa enviar dados

Resposta que o Spring devolve

- Status: 200 — deu certo
- Corpo: a lista em JSON

POST: envia dados no corpo

Exemplo: cadastrar a aluna Ana.

Pedido

- Verbo: POST
- Endereço: /api/alunos
- Corpo:

```
{"nome": "Ana", "turma": "Entra21"}
```

Resposta

Status 200 e o aluno já com id, em JSON.

O que é o corpo — o body

Corpo = o recado que vai **dentro** do pedido, não na URL.

- Na Aula 02, o nome ia na URL:
`/api/saudacao?nome=Ana`
- Hoje, no cadastro, nome e turma vão **dentro** do pedido, em JSON. Isso é o corpo.

A URL diz *para quem* estamos falando `/api/alunos`.
O corpo diz *o que* queremos gravar.

O Axios monta essa carta para a gente

Quando escrevemos no JavaScript:

```
axios.post("/api/alunos", { nome: "Ana", turma: "Entra21"})
```

O Axios faz, por baixo:

- 1 Verbo POST
- 2 Endereço /api/alunos
- 3 Corpo = aquele objeto, transformado em JSON

No Java, @RequestBody significa:

“pega o JSON que veio no corpo e monta um objeto”.

Como ver isso no navegador

Apertem **F12** → aba **Network** / Rede.

Cliquem na linha do pedido alunos:

- **Headers** — verbo e endereço
- **Payload** ou **Request** — o corpo que **mandamos**
- **Response** — o corpo que o Spring **devolveu**

Sempre que algo “não gravar”, olhem essa aba antes de mudar o código de novo.

As quatro ações de um cadastro

Ação	Verbo HTTP	Exemplo
Listar / ler	GET	Ver os alunos
Cadastrar	POST	Incluir a Ana
Editar	PUT	Trocar a turma da Ana
Excluir	DELETE	Apagar a Ana

CRUD é só o nome disso:

Create, Read, Update, Delete.

A lista precisa ser a mesma o tempo todo

Imaginem um caderno de chamada.

- GET = ler o caderno
- POST = escrever um nome novo **nesse** caderno
- PUT = corrigir um nome
- DELETE = apagar um nome

Se a cada leitura vocês pegam um caderno novo em branco, o nome que acabaram de escrever some.

No código, o caderno é a **lista**.

Ela precisa ficar **na classe do Controller**, não dentro do método que lista.

Errado: lista dentro do método

Cada GET cria uma lista nova e joga fora no fim.

```
@GetMapping("/api/alunos")
public List<Aluno> listar() {
    List<Aluno> lista = new ArrayList<>();
    lista.add(new Aluno(1, "Ana", "Entra21"));
    return lista;
}
```

O POST até pode adicionar um aluno...
mas no próximo GET essa lista já não existe.

Certo: lista na classe, métodos só usam

A lista continua **no Controller**.

Só sai de **dentro da função** e vira campo da classe.

```
@RestController
public class AlunoController {

    private List<Aluno> alunos = new ArrayList<>();

    @GetMapping("/api/alunos")
    public List<Aluno> listar() {
        return alunos;
    }
}
```

GET, POST, PUT e DELETE mexem nessa **mesma** lista.

Vamos criar um projeto novo

Primeira parte da aula: projeto **cadastro-alunos**.

Serve para treinar CRUD com calma,
e relembrar o `start.spring.io`.

O que esse projeto vai ter no final da Parte 1

Uma página com formulário e lista de alunos.

Dá para cadastrar. Editar e excluir ficam nos desafios.

Passo 1 — Gerar o projeto

Entrem em `https://start.spring.io`

- Project: Maven Language: Java
- Group: `com.entra21`
- Artifact: `cadastro-alunos`
- Package name: `com.entra21.cadastro`
- Packaging: Jar Java: 21
- Dependency: **Spring Web**

Generate → baixem → extraiam → abram a pasta no VS Code.

Passo 2 — Conferir se sobe

No terminal, dentro da pasta do projeto:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Se aparecer que o servidor está na porta 8080, está ok.

Por enquanto não tem página nossa — isso é o próximo passo.

Dois Spring ao mesmo tempo

Se o StockSales ainda estiver rodando, a porta 8080 já está ocupada. Parem o outro antes.

Quais arquivos vamos criar

Projeto: **cadastro-alunos**

Arquivo	Para quê
<code>Aluno.java</code>	Os dados de um aluno
<code>AlunoController.java</code>	A API: listar e cadastrar
<code>static/index.html</code>	A página
<code>static/js/app.js</code>	O JavaScript da página

A classe `Application` o site já criou.

Não precisamos mexer nela.

Onde cada arquivo fica

```
cadastro-alunos/  
  src/main/java/com/entra21/cadastro/  
    CadastroAlunosApplication.java    (ja existe)  
    Aluno.java                        (vamos criar)  
    AlunoController.java              (vamos criar)  
  src/main/resources/static/  
    index.html                       (vamos criar)  
    js/app.js                        (vamos criar)
```

Java fica em java/.

Página e JS ficam em resources/static/.

Onde estamos

Projeto **cadastro-alunos**

Pasta `src/main/java/com/entra21/cadastro/`

Arquivo novo: `Aluno.java`

Esta classe só guarda dados: id, nome e turma.

Ainda não é a API.

Aluno.java — atributos e construtores

Onde estamos

Aluno.java

```
package com.entra21.cadastro;

public class Aluno {
    private int id;
    private String nome;
    private String turma;

    public Aluno() {
    }

    public Aluno(int id, String nome, String turma) {
        this.id = id;
        this.nome = nome;
        this.turma = turma;
    }
}
```


Por que dois construtores

- `Aluno()` — o Spring usa quando chega um JSON. Ele cria o objeto vazio e chama os set.
- `Aluno(id, nome, turma)` — nós usamos para colocar Ana, Bruno e Carla na lista inicial.

Sem o construtor vazio, o cadastro costuma quebrar com erro no terminal.

Passo 4 — Começar o Controller

Onde estamos

Projeto **cadastro-alunos**

Mesma pasta do `Aluno.java`

Arquivo novo: `AlunoController.java`

Aqui fica a API. E aqui fica a lista —
na classe, não dentro do método `listar`.

AlunoController.java — a lista

Onde estamos

AlunoController.java

```
package com.entra21.cadastro;

@RestController
public class AlunoController {

    private List<Aluno> alunos = new ArrayList<>();
    private int proximoId = 1;

    public AlunoController() {
        alunos.add(new Aluno(1, "Ana", "Entra21"));
        alunos.add(new Aluno(2, "Bruno", "Entra21"));
        alunos.add(new Aluno(3, "Carla", "Entra21"));
        proximoId = 4;
    }
}
```

GET — só devolve a lista da classe

Onde estamos

Ainda em `AlunoController.java`,
um método novo dentro da classe

```
@GetMapping("/api/alunos")  
public List<Aluno> listar() {  
    return alunos;  
}
```

Salvem, reiniciem o Spring, abram no navegador:

`http://localhost:8080/api/alunos`

Tem que aparecer o JSON de Ana, Bruno e Carla.
Ainda sem página — só testando a API.

POST — lê o corpo e adiciona na lista

Onde estamos

Ainda em AlunoController.java

```
@PostMapping("/api/alunos")
public Aluno cadastrar(@RequestBody Aluno aluno) {
    aluno.setId(proximoId);
    proximoId = proximoId + 1;
    alunos.add(aluno);
    return aluno;
}
```

@RequestBody = pega o JSON do **corpo** do pedido.

Imports novos: PostMapping e RequestBody.

O que o POST faz, em ordem

- 1 O JavaScript manda JSON no corpo:
nome e turma
- 2 O Spring monta um Aluno com os set
- 3 Nós colocamos o próximo id
- 4 Adicionamos na lista **da classe**
- 5 Devolvemos o aluno já com id

O GET seguinte vê o aluno novo,
porque a lista não foi recriada.

Onde estamos

Projeto **cadastro-alunos**

Pasta `src/main/resources/static/`

Arquivo novo: `index.html`

Esta é a tela. Ainda não tem lógica — só o desenho:
um formulário e uma lista vazia.

Onde estamos

src/main/resources/static/index.html

```
<h1>Cadastro de alunos</h1>

<form id="form-aluno">
  <input id="nome" type="text" placeholder="Nome">
  <input id="turma" type="text" placeholder="Turma">
  <button type="submit">Cadastrar</button>
</form>

<ul id="lista-alunos"></ul>

<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
<script src="/js/app.js"></script>
```

Axios **antes** do app.js.

Onde estamos

Projeto **cadastro-alunos**

Pasta `src/main/resources/static/js/`

Arquivo novo: `app.js`

Duas tarefas neste arquivo:

- 1 Ao abrir a página, buscar a lista na API
- 2 Ao enviar o formulário, cadastrar na API e atualizar a lista

Onde estamos

static/js/app.js

```
var lista = document.getElementById("lista-alunos");

function carregarAlunos() {
  axios.get("/api/alunos")
    .then(function (resposta) {
      var alunos = resposta.data;
      lista.innerHTML = "";
      for (var i = 0; i < alunos.length; i++) {
        var li = document.createElement("li");
        li.textContent =
          alunos[i].nome + " - " + alunos[i].turma;
        lista.appendChild(li);
      }
    });
}
```

Por que uma função carregarAlunos

Vamos chamar essa função em dois momentos:

- Quando a página abre
- Depois de cadastrar — para a lista mostrar o novo

Se cadastrar e não chamar de novo, o backend gravou, mas a tela fica antiga.

Onde estamos

Ainda em `static/js/app.js`, abaixo do `listar`

O formulário, sozinho, recarrega a página.
`preventDefault` impede isso.

```
var form = document.getElementById("form-aluno");

form.addEventListener("submit", function (evento) {
  evento.preventDefault();
  var nome = document.getElementById("nome").value;
  var turma = document.getElementById("turma").value;

  axios.post("/api/alunos", { nome: nome, turma: turma })
    .then(function () {
      carregarAlunos();
    });
});
```

O caminho do cadastro, do clique até a tela

- 1 A pessoa preenche nome e turma e clica em Cadastrar
- 2 O JS manda POST /api/alunos com JSON no corpo
- 3 O Controller lê o corpo, gera o id, coloca na lista
- 4 O JS recebe a resposta e chama carregarAlunos
- 5 O GET busca a lista de novo e redesenha a

No F12, aba Network, vocês devem ver um POST e, em seguida, um GET.

Conferindo o exemplo juntos

`http://localhost:8080/` abre o formulário

Ana, Bruno e Carla aparecem

Cadastrar um nome novo atualiza a lista

Recarregar a página **mantém** o nome novo

Parar e subir o Spring de novo: voltam só os três

Se o nome some ao recarregar a página,
a lista ainda está sendo criada dentro do `listar`.

Erros que mais aparecem

- Porta 8080 ocupada — outro Spring ainda rodando
- `axios is not defined` — script do Axios depois do `app.js`
- POST quebra — faltou construtor vazio no `Aluno`
- Página recarrega e o form esvazia — faltou `preventDefault`
- Cadastrou e a lista não muda — não chamou `carregarAlunos`

Agora é com vocês

Continuem no projeto **cadastro-alunos**.
Mesmos arquivos. Não criem outro projeto.

- 1 Excluir aluno
- 2 Editar no backend
- 3 Editar pela tela

Salve → reinicie o Spring → teste no navegador.
F12, aba Network.

PUT e DELETE usam o id no endereço

Para mudar **um** aluno, o servidor precisa saber **qual**.

Ação	Pedido
Editar o aluno 2	PUT /api/alunos/2 corpo com nome e turma novos
Excluir o aluno 2	DELETE /api/alunos/2 sem corpo

O 2 é o `@PathVariable`.

Onde estamos

- 1) `AlunoController.java`
- 2) `static/js/app.js`

No Controller, método novo:

```
DELETE /api/alunos/{id}
```

Percorram a lista. Se o id bater: `alunos.remove(i)` e parem o `for`.

No JS, cada item da lista ganha um botão Excluir.

No clique: `axios.delete("/api/alunos/" + id)`
e depois `carregarAlunos()`.

Desafio 1 — o botão no JS

Onde estamos

static/js/app.js, dentro do for
que monta a lista

```
var botao = document.createElement("button");
botao.textContent = "Excluir";
botao.setAttribute("data-id", alunos[i].id);

botao.addEventListener("click", function (evento) {
    var id = evento.target.getAttribute("data-id");
    axios.delete("/api/alunos/" + id)
        .then(function () {
            carregarAlunos();
        });
});

li.appendChild(botao);
```

Desafio 2 — Editar no Controller

Onde estamos

AlunoController.java

Método novo: PUT /api/alunos/{id}

Recebam o id da URL e o aluno do corpo.

Achem na lista, atualizem nome e turma, devolvam o aluno.

```
@PutMapping("/api/alunos/{id}")
public Aluno atualizar(@PathVariable int id,
                      @RequestBody Aluno dados) {
    // for na lista, se o id bater:
    // setNome e setTurma, depois return
    return null;
}
```

Desafio 3 — Editar pela tela

Onde estamos

`static/js/app.js`

Objetivo: clicar em Editar, o formulário se preenche, e o Cadastrar passa a salvar a alteração.

- 1 Variável `idEditando`, começando vazia
- 2 Botão Editar coloca o id, o nome e a turma no form
- 3 Se `idEditando` estiver vazio → POST
- 4 Se tiver id → PUT para aquele id
- 5 No fim, limpem o form e o `idEditando`

Se terminarem os três, ainda no **cadastro-alunos**:

- Não cadastrar se o nome estiver vazio
- Mostrar o id na lista
- Texto na página: “Aluno cadastrado”

Checklist

Estou no projeto cadastro-alunos?

A lista é campo da classe, não do método listar?

Aluno tem construtor vazio?

POST usa @RequestBody?

O form tem preventDefault?

Depois de gravar, chama carregarAlunos?

Retomamos às **20h20**

Fechem o cadastro-alunos.
Na segunda parte abrimos o **StockSales**.

Mudamos de projeto

Onde estamos

Agora é o projeto **StockSales**,
não o cadastro-alunos.

Parem o Spring do cadastro-alunos.

Abram a pasta do StockSales e subam de novo.

Onde o StockSales parou

Arquivos que vocês já têm:

- `controller/ProdutoController.java`
- `model/Produto.java`
- `static/produtos.html`
- `static/js/produtos.js`

Hoje a lista é criada **dentro** de `listarProdutos`.

Por isso todo GET volta sempre os quatro produtos fixos.

A lista deve **continuar no Controller**.

Só precisa sair de dentro dessa função.

O que vamos fazer no StockSales

- 1 Subir a lista de produtos para a classe ProdutoController
- 2 Cadastrar, editar e excluir produto
- 3 Formulário na tela de produtos
- 4 Criar cliente: classe, Controller e tela

É o mesmo raciocínio do cadastro-alunos.
Só mudam os campos e os nomes dos arquivos.

Produto.java — construtor vazio

Onde estamos

Projeto **StockSales**

Arquivo `src/main/java/com/entra21/stocksales/model/Produto.java`

Acrescentem este construtor. Mantenham o outro.

```
public Produto() {  
}
```

Sem ele, o POST de produto quebra.

A lista sai do método, fica na classe

Onde estamos

controller/ProdutoController.java

Não tirem a lista do Controller.

Tirem só de dentro de listarProdutos.

```
private List<Produto> produtos = new ArrayList<>();
private int proximoId = 1;

public ProdutoController() {
    produtos.add(new Produto(1, "Teclado Mecanico", 250, 12));
    produtos.add(new Produto(2, "Mouse Gamer", 120, 30));
    produtos.add(new Produto(3, "Monitor 24", 900, 8));
    produtos.add(new Produto(4, "Headset", 200, 15));
    proximoId = 5;
}
```

listarProdutos fica só isto

Onde estamos

Ainda em `ProdutoController.java`

Antes o método criava a lista. Agora só devolve.

```
@GetMapping("/api/produtos")  
public List<Produto> listarProdutos() {  
    return produtos;  
}
```

Testem `/api/produtos` no navegador:
tem que continuar mostrando os quatro.

POST de produto

Onde estamos

Ainda em `ProdutoController.java`

```
@PostMapping("/api/produtos")
public Produto cadastrar(@RequestBody Produto produto) {
    produto.setId(proximoId);
    proximoId = proximoId + 1;
    produtos.add(produto);
    return produto;
}
```

PUT e DELETE: mesmo for do desafio dos alunos,
com `/api/produtos/{id}`.

Onde estamos

`static/produtos.html` e `static/js/produtos.js`

Na HTML, acima da lista: formulário com nome, preço e estoque.

No JS:

- Função `carregarProdutos` — vocês já listam; só extraíam para uma função
- `preventDefault` no form
- `axios.post("/api/produtos", { ... })`
- Botão Excluir em cada item

Preço e estoque são número

Onde estamos

static/js/produtos.js

O input sempre devolve texto. O Java espera número.

```
axios.post("/api/produtos", {  
  nome: document.getElementById("nome").value,  
  preco: Number(document.getElementById("preco").value),  
  estoque: Number(document.getElementById("estoque").value)  
});
```

Classe Cliente

Onde estamos

Projeto **StockSales**

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Cliente.java

```
public class Cliente {  
    private int id;  
    private String nome;  
    private String email;  
    private String telefone;  
  
    public Cliente() {}  
}
```

Construtor com os quatro valores + gets e sets.

Onde estamos

Arquivos novos:

`controller/ClienteController.java`

`static/clientes.html`

`static/js/clientes.js`

Copiem o esquema do produto:

- Lista na **classe** do Controller
- GET / POST / PUT / DELETE em `/api/clientes`
- Formulário: nome, e-mail, telefone

Na home, um link para `/clientes.html`.

O que não entra hoje

- Vendas e baixa de estoque — próxima aula
- Banco de dados
- Login

Meta no StockSales

Cadastrar, listar, editar e excluir produtos.

Cadastrar, listar, editar e excluir clientes.

Lista na classe do Controller, ainda sem banco.

Produto novo aparece na lista

Recarregar a página mantém, até reiniciar o Spring

Excluir some da lista

Editar muda nome, preço ou estoque

Clientes no mesmo esquema

Home leva a produtos e a clientes

O que vimos hoje

- 1 Requisição: verbo + endereço + corpo
- 2 Resposta: status + JSON
- 3 POST/PUT mandam dados no corpo
- 4 A lista fica no Controller, fora do método de listar
- 5 CRUD no cadastro-alunos e no StockSales

Aula 04

- Vendas no StockSales
- Escolher cliente e produtos
- Baixa de estoque

Tragam o StockSales com produtos e clientes cadastráveis.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21